

Comparative Analysis and Strategic Roadmap for Next-Generation Blockchain E-Voting Systems

Executive Summary

The digitalization of democratic processes represents one of the most significant challenges and opportunities in modern governance. As traditional voting infrastructures grapple with issues of transparency, trust, and logistical inefficiency, blockchain technology has emerged as a compelling solution to ensure immutable, verifiable, and secure elections. However, the transition from theoretical utility to practical implementation is fraught with architectural trade-offs known as the "E-Voting Trilemma": the difficulty of simultaneously achieving Security (integrity), Privacy (anonymity), and Scalability (efficiency).

This comprehensive report provides an exhaustive analysis of three distinct research contributions to this field:

1. **"E-Voting via Upgradable Smart Contracts on Blockchain"** (Paper A).¹
2. **"Secured Electronic Voting System using Blockchain Technology"** (Paper B).¹
3. **"A Secure and Transparent Blockchain-driven E-Voting System using Smart Contract and IPFS-based Scalability"** (Paper C).¹

These papers represent "Generation 1" blockchain voting solutions. They successfully demonstrate how Distributed Ledger Technology (DLT) can prevent ballot stuffing and retroactive data manipulation. Paper A pioneers the use of **upgradable proxy patterns** to solve the rigidity of smart contract immutability. Paper B focuses on **accessibility and frontend optimization**, bridging the gap between complex chain logic and user experience. Paper C addresses the critical economic barrier of gas costs by introducing a **hybrid storage model** leveraging the InterPlanetary File System (IPFS).¹

However, a rigorous critique reveals that these solutions fundamentally rely on semi-centralized authentication bridges (Aadhaar APIs, Admin Whitelists) and lack robust coercion resistance. They operate on the assumption that the voter is free from physical intimidation and that the central identity oracle is uncompromised.

To bridge the gap between these academic prototypes and a viable nation-scale voting system, this report proposes a "Generation 2" architecture—the **Aegis Protocol**. This proposed framework integrates **Zero-Knowledge Proofs (ZKPs)** via Minimal Anti-Collusion Infrastructure (MACI) to ensure receipt-freeness; **Self-Sovereign Identity (SSI)** via Anon Aadhaar to decouple verification from surveillance; **Account Abstraction (ERC-4337)** to eliminate the user friction of gas fees; and **Layer-2 ZK-Rollups** to achieve the transaction

throughput required for mass elections.

1. Introduction: The Crisis of Trust and the Blockchain Promise

1.1 The Erosion of Electoral Integrity

Elections are the bedrock of democratic legitimacy. Yet, globally, confidence in electoral outcomes is waning. Traditional paper-based systems, while auditable, are susceptible to ballot stuffing, booth capturing, and logistical errors during counting.¹ Direct Recording Electronic (DRE) systems (EVMs) have improved counting speed but introduced "black box" vulnerabilities, where the internal logic is opaque to the voter, and the chain of custody for the hardware can be broken.²

The core issue is **trust**. In a centralized system, the voter must trust the election commission, the software vendors, the database administrators, and the physical security of the storage facilities. If any single link in this chain is compromised, the integrity of the entire election collapses.

1.2 The Blockchain Paradigm Shift

Blockchain technology inverts this trust model. By utilizing a decentralized, immutable ledger, it removes the need to trust a central authority to maintain the count. Instead, the voter trusts the **cryptographic proofs** and the **consensus mechanism** of the network.

- **Immutability:** Once a vote is included in a block and confirmed by the network (e.g., Ethereum), it cannot be modified or deleted without overwhelming computational resources (51% attack).³
- **Transparency:** The logic of the election—who can vote, how votes are tallied—is public code (Smart Contracts), auditable by anyone.
- **Decentralization:** The ledger is replicated across thousands of nodes, eliminating single points of failure and censorship.⁴

1.3 The E-Voting Trilemma

Despite these advantages, implementing blockchain voting is not merely a matter of deployment. Architects must navigate the E-Voting Trilemma:

1. **Integrity (Security):** The system must ensure only eligible voters vote, and only once. (Addressed by all papers via Smart Contracts and Identity checks).
2. **Privacy (Anonymity):** The system must protect the secrecy of the ballot. (A challenge for public blockchains like Ethereum where transactions are visible).
3. **Scalability (Efficiency):** The system must handle millions of transactions without

prohibitive costs or latency. (A major bottleneck for Ethereum Layer 1).

The three papers under review offer distinct strategies to solve specific vertices of this triangle, often at the expense of others.

2. Deep Dive Analysis: Paper A

Title: E-Voting via Upgradable Smart Contracts on Blockchain **Source ID:** ¹

2.1 Summary and Core Philosophy

Paper A addresses a practical, often overlooked aspect of blockchain software engineering: **software lifecycle management**. In standard blockchain development, smart contracts are immutable once deployed. If a bug is discovered in the vote-counting function, or if the election commission needs to alter the rules (e.g., extending the voting deadline), a standard contract cannot be changed. This rigidity poses a catastrophic risk for critical infrastructure like national elections.¹

The authors propose an architecture centered on **Upgradable Smart Contracts** using the **Proxy Pattern** on the Ethereum blockchain. This approach prioritizes **maintainability** and **longevity**, acknowledging that election systems are not static and require iterative improvements.

2.2 Uniqueness: The Proxy Pattern Implementation

The defining feature of Paper A is the implementation of the **Transparent Proxy Pattern** (facilitated by the OpenZeppelin plugin).

- **Mechanism:** The architecture separates the system into two contract types:
 - **Proxy Contract:** Holds the state (storage) of the election (e.g., vote counts, voter registry). It exposes the user-facing API.
 - **Implementation Contract (Logic):** Contains the Solidity code for functions like `vote()` and `addCandidate()`.
- **Delegatecall:** When a user sends a transaction to the Proxy, the Proxy uses the EVM opcode `delegatecall` to execute the code of the Implementation Contract *in the context* of the Proxy. This means the Logic contract runs, but it reads and writes to the Proxy's storage.¹
- **The Upgrade:** To upgrade, the Election Manager deploys a new Implementation Contract (ElectionV2) and updates the Proxy to point to this new address. The Proxy retains all previous vote data (state) but now processes new transactions using the V2 logic.

Strategic Advantage: This allows for bug fixes and feature additions (e.g., adding a "None of the Above" option mid-cycle) without disrupting the user experience or migrating data, a

critical feature for government-grade software.

2.3 Authentication Module: Biometric Fusion

The paper employs a multi-factor authentication (MFA) regime:

- **Factors:** Email-ID, Aadhaar Registration ID, and **Facial Scan**.
- **Process:** The frontend captures a facial scan and sends it to a backend (likely Python/Flask, though the paper is vague on the backend specifics beyond mentioning "Flask framework" for face detection in related work contexts).
- **Verification:** This biometric data is verified against the "existing E-Aadhaar database".¹
- **Weakness:** The reliance on a Flask backend to process facial recognition introduces a centralized point of failure. If the server processing the face matching is compromised, an attacker can bypass the biometric check and submit transactions directly to the smart contract if they possess the user's private key.

2.4 User Experience: Gas Station Network (GSN)

Paper A explicitly addresses the "Gas Friction" problem. In Ethereum, every state-changing transaction (like casting a vote) requires a fee paid in Ether. Expecting voters to acquire cryptocurrency is a non-starter for adoption.

- **GSN Solution:** The system integrates a **Gas Station Network**. Voters sign "meta-transactions" (messages containing their vote intent). These are sent to GSN Relayers, who pay the gas fee to submit the transaction to the blockchain. The election commission funds the Relayers.¹
- **Impact:** This provides a "Web2-like" experience where the user clicks "Vote" and the cost is abstracted away, significantly enhancing accessibility.

2.5 Critical Weaknesses and Disadvantages

- **Centralization of Upgrade Power:** The "Election Manager" holds the keys to the Proxy. A compromised manager could upgrade the contract to a malicious version that fundamentally alters the tallying logic or freezes the election. The paper does not discuss putting this admin capability under a multi-signature (Multisig) or DAO governance structure.⁵
 - **Privacy Leakage:** While the paper mentions "anonymity," storing votes on a public Ethereum ledger (even via proxy) allows anyone to perform linkability analysis. If a user's Ethereum address is linked to their identity (which it is, via the KYC process), their vote is effectively public.
 - **Flask Bottleneck:** The facial recognition runs on a centralized server. This reintroduces the "black box" risk—voters must trust the server to accurately match faces and not store their biometric data for nefarious purposes.
-

3. Deep Dive Analysis: Paper B

Title: Secured Electronic Voting System using Blockchain Technology **Source ID:** ¹

3.1 Summary and Core Philosophy

Paper B shifts the focus from backend architecture to **Application Layer Efficiency** and **Accessibility**. The authors argue that the primary barrier to e-voting adoption is not just security, but usability and interface responsiveness. The paper presents a system built on the **Ethereum** blockchain but places heavy emphasis on the **React JS** frontend framework to ensure a seamless voter experience.

3.2 Uniqueness: Frontend Optimization Strategy

The paper devotes significant analysis to the choice of web framework, comparing **React JS** against **Angular JS**.

- **Virtual DOM:** The authors select React JS for its use of the Virtual DOM, which optimizes rendering performance. In an election scenario with millions of concurrent users, client-side rendering efficiency is crucial to prevent the application from becoming unresponsive.¹
- **Component Reusability:** The modular nature of React allows for the creation of reusable voting components (e.g., Candidate Cards, Verification Modals), streamlining development and testing.
- **Significance:** While this seems like a standard web development choice, in the context of blockchain dApps (which are notoriously slow due to block confirmation times), a highly responsive frontend is essential to maintain user confidence during the "pending transaction" state.

3.3 Authentication Module: The Admin Whitelist

Unlike Paper A's biometric fusion, Paper B relies on a **Centralized Admin Model** for authentication.

- **Process:** The Admin is responsible for verifying voter credentials (off-chain) and "registering" the voter on the blockchain.
- **Mechanism:** This likely involves the Admin calling a registerVoter(address voter) function on the smart contract, which sets a boolean flag (e.g., isRegistered[address] = true).
- **Metamask Integration:** The voter authenticates locally using **Metamask**. The wallet address serves as the user's digital identity.¹
- **Analysis:** This is a regression in security compared to Paper A. It places absolute trust in the Admin. A corrupt Admin can arbitrarily exclude voters (voter suppression) or register Sybil accounts (ballot stuffing) without any cryptographic proof of the voter's actual eligibility provided to the chain.

3.4 Discrepancy Analysis: "Machine Learning" and "Hyperledger"

A detailed review of Paper B reveals significant inconsistencies indicative of a broader scope that was not fully implemented or edited out.

- **Phantom Keywords:** The paper includes "Machine Learning" and "Data Science" in its keywords.¹ However, the text contains *no implementation* or even theoretical discussion of ML models (e.g., for fraud detection or pattern recognition). This suggests the keywords may have been aspirational or misapplied.
- **Platform Confusion:** The abstract and introduction heavily focus on **Ethereum**. However, Section II mentions **Hyperledger Sawtooth** as a proposed platform for a "trustworthy, open, and distributed online voting system".¹ This contradiction weakens the architectural clarity—is the system Proof-of-Work (Ethereum) or Proof-of-Elapsed-Time (Sawtooth)? The implementation details (Metamask, Solidity) point to Ethereum, making the Sawtooth reference a likely artifact of the literature review rather than the proposed solution.

3.5 Critical Weaknesses and Disadvantages

- **Single Point of Failure (Admin):** The reliance on an Admin to manually or programmatically whitelist voters defeats the purpose of a decentralized, trustless system.
- **Lack of Privacy:** The paper admits that "privacy... [is] not the primary focus".⁴ On a public Ethereum chain, without advanced encryption (like ZKPs), every vote is traceable to the voting address. Since the Admin knows which address belongs to which voter, the ballot is not secret.
- **Cost Barrier:** Unlike Paper A, this paper does not mention a Gas Station Network. It implies voters must use Metamask to "approve the transaction," suggesting they must hold ETH to pay for gas, which is a major barrier to entry for the general public.

4. Deep Dive Analysis: Paper C

Title: A Secure and Transparent Blockchain-driven E-Voting System using Smart Contract and IPFS-based Scalability **Source ID:** ¹

4.1 Summary and Core Philosophy

Paper C represents the most architecturally mature attempt to solve the **Scalability** and **Cost** vertices of the trilemma. The authors correctly identify that storing large amounts of data (voter demographics, biometric hashes, full ballot details) on the Ethereum blockchain is economically infeasible due to gas costs. Their solution is a **Hybrid Storage Architecture** leveraging the **InterPlanetary File System (IPFS)**.

4.2 Uniqueness: IPFS-Based Scalability

The integration of IPFS is the paper's standout contribution.

- **The Problem:** Storing a 1KB string on Ethereum can cost ~\$5–\$10 in gas fees (depending on network congestion). For a national election with millions of voters, this linear cost scaling is prohibitive.
- **The Solution:** The system encrypts the voter data and ballot information and uploads it to IPFS (a decentralized peer-to-peer storage network). IPFS returns a **Content Identifier (CID)**, a cryptographic hash (e.g., QmX...) that acts as a permanent address for that data.¹
- **On-Chain Optimization:** Only this 32-byte CID is stored on the Ethereum smart contract. The gas cost becomes constant and minimal, regardless of the size of the underlying data.
- **Impact:** This dramatically reduces the cost per vote, making the system potentially viable for large-scale deployments.

4.3 Authentication Module: The "Digital Voter ID"

Paper C introduces a sophisticated **3-Factor Authentication** flow:

1. **Aadhaar API:** First, the user proves their identity using the centralized government API.
2. **Twilio OTP:** Upon success, a mobile OTP is verified, proving possession of the registered device.
3. **Digital Voter ID:** The system generates a unique "Digital Voter ID" (likely a wallet pair or a specialized token) that is linked to the IPFS CID of their registration data.¹
4. **Flow:** When voting, the user logs in with the Digital Voter ID. The smart contract retrieves the IPFS CID, verifies the hash matches the on-chain record, and allows the vote.

4.4 Security: Encryption and Hashing

Unlike Paper B, Paper C takes data privacy seriously.

- **AES-256:** Voter metadata stored on IPFS is encrypted using AES-256. This is critical because IPFS is a public network; without encryption, anyone with the CID could read the voter's personal info.
- **SHA-256:** The system uses SHA-256 hashing to ensure data integrity before it is anchored to the blockchain.¹

4.5 Performance Metrics

Paper C provides concrete performance data (a rarity in theoretical papers):

- **Voter Registration Latency:** ~1.45 seconds.
- **Vote Casting Latency:** ~1.21 seconds.¹
- **Gas Analysis:** The paper explicitly claims a "significant reduction in transaction costs"

compared to full on-chain storage systems like DVTChain.¹

4.6 Critical Weaknesses and Disadvantages

- **IPFS Persistence:** The paper does not address the issue of **IPFS Garbage Collection**. Data on IPFS is not permanent unless "pinned" by a node. If the election nodes go offline or stop pinning the data, the CIDs stored on the blockchain become broken links, leading to data loss.⁶
- **Encryption Key Management:** While AES-256 is strong, the paper does not detail *who* holds the decryption keys. If the Election Authority holds them, they can deanonymize every voter (violating ballot secrecy). If the voter holds them, and loses them, their vote cannot be verified.
- **Aadhaar Dependence:** Like Paper A, it relies on the Aadhaar API, inheriting the "Oracle Problem." If the API is down or compromised, the blockchain registration fails.

5. Comparative Matrix and Superiority Analysis

This section juxtaposes the three papers across critical functional modules to determine superiority.

Feature / Module	Paper A	Paper B	Paper C	Superior Implementation
Smart Contract Architecture	Upgradable (Proxy Pattern)	Immutable (Standard)	Immutable (Standard)	Paper A (Crucial for maintenance and bug patching)
Scalability & Storage	On-Chain / GSN	On-Chain + SQL DB	IPFS + On-Chain	Paper C (Lowest gas cost and decentralized storage)
Authentication Strength	2FA (Face + Aadhaar)	1FA (Wallet + Admin)	3FA (Aadhaar + OTP + Wallet)	Paper C (Multi-layered defense)

				against impersonation)
User Experience (Gas)	Gas Station Network	User Pays Gas	IPFS Offloading (Low Gas)	Paper A (GSN completely removes user cost friction)
Frontend Strategy	React (Basic)	React vs Angular Analysis	DApp Interface	Paper B (Best consideration of client-side performance)
Privacy / Encryption	Standard	None Specified	AES-256 + SHA-256	Paper C (Best data protection implementation)

5.1 Superiority Verdicts

- **Superior in Flexibility: Paper A.** The Proxy Admin pattern is the only architecture that acknowledges software realities (bugs, updates).
- **Superior in Scalability: Paper C.** The IPFS integration is the only architecture that could theoretically scale to a national election without bankrupting the organizers in gas fees.
- **Superior in UX: Paper A.** The Gas Station Network is the gold standard for onboarding non-crypto-native users.

6. The "Aegis" Protocol: An Innovative Architecture for Next-Gen E-Voting

The analysis of Papers A, B, and C reveals a "Generation 1" plateau: they solve Integrity but fail on Privacy and Coercion Resistance. They rely on "security through obscurity" (Paper B) or "trusted hardware/APIs" (Paper A/C).

To satisfy the unsatisfied requirements of **true ballot secrecy** and **coercion resistance**, we propose the **Aegis Protocol**, a web application architecture that integrates techniques not covered in the reviewed papers.

6.1 Innovation 1: Minimal Anti-Collusion Infrastructure (MACI)

Problem: In Papers A, B, and C, if a coercer demands to see your screen or your private key, they can verify how you voted. This enables vote-buying. **Solution:** Implement **MACI**.⁷

- **Mechanism:** Users vote by signing a message with their private key and encrypting it with the Coordinator's public key.
- **Key Rotation:** The core innovation is that users can *change their key* at any time by sending a signed command.
- **The Anti-Coercion Logic:** A voter can be coerced into using Key A to cast a vote for a Dictator. Later (or earlier), the voter sends a "Key Change" command to Key B. The smart contract (and Coordinator) invalidates Key A. The Coordinator processes messages in reverse order. The vote cast with Key A is discarded as "invalid key," and the vote cast with Key B counts. The coercer *cannot know* that Key A was invalidated because the state update is encrypted (Zero-Knowledge).
- **Result:** Receipt-freeness. The voter cannot prove to the coercer which vote counted.

6.2 Innovation 2: Self-Sovereign Identity (SSI) with Anon Aadhaar

Problem: All three papers ping the Aadhaar API directly. This creates a metadata trail: The government knows *who* voted and *when*. **Solution:** Use **Anon Aadhaar** with **Zero-Knowledge Proofs (ZKPs)**.⁹

- **Mechanism:** The Aadhaar card contains a secure QR code signed by the UIDAI. The user scans this QR code locally on their device.
- **ZK Circuit:** The app generates a ZK-proof that asserts: *"I possess a valid Aadhaar signature, and I am over 18, and I reside in District X."*
- **Nullifier:** The circuit outputs a unique "Nullifier" hash (derived from the data but mathematically irreversible).
- **Submission:** The user submits the Proof + Nullifier to the blockchain. The contract verifies the proof and checks if the Nullifier has been used.
- **Result:** The blockchain knows the user is a valid citizen, but *not which citizen*. The UIDAI API is never queried during the vote, preserving privacy from the state.

6.3 Innovation 3: Account Abstraction (ERC-4337)

Problem: Paper B requires Metamask. Paper A uses GSN (older tech). **Solution:** Implement **ERC-4337 Smart Accounts**.¹¹

- **Bundlers & Paymasters:** The application uses a "Paymaster" contract. When a user votes, the Paymaster validates the ZK-proof (from Innovation 2). If valid, the Paymaster "sponsors" the transaction, paying the gas in ETH.
- **Social Login:** The user's "key" can be a Passkey (FaceID/TouchID) stored on their phone's secure enclave, wrapped in a WebAuthn signature. The Smart Account verifies this signature.
- **Result:** The user experiences a "gasless," "seedless" flow. No private keys to lose, no ETH to buy.

6.4 Innovation 4: Layer-2 ZK-Rollups for Hyper-Scalability

Problem: Even with IPFS (Paper C), Ethereum L1 can only process ~15-30 TPS. A national election needs 10,000+ TPS. **Solution:** Deploy the voting contracts on a **ZK-Rollup** (e.g., Scroll, zkSync).¹³

- **Mechanism:** The Rollup executes thousands of votes off-chain and generates a "Validity Proof" (SNARK/STARK). This proof is submitted to Ethereum L1.
- **Benefit:** This inherits Ethereum's security but reduces costs by ~100x and increases throughput by ~1000x. It solves the scalability limits of Paper C while maintaining the decentralization missing in Paper B.

6.5 Innovation 5: Soulbound Tokens (SBTs) for Reputation & Registration

Problem: Paper B's whitelist is opaque. **Solution:** Issue **Soulbound Tokens (SBTs)** upon registration.¹⁵

- **Mechanism:** When the Anon Aadhaar proof is verified, the contract mints a non-transferable SBT to the user's Smart Account.
- **Voting Gate:** The Voting Contract checks `balanceOf(user, VoterSBT) == 1`.
- **Quadratic Voting:** Since we have Sybil resistance via SBTs, we can implement **Quadratic Voting**, allowing voters to express the *intensity* of their preference (e.g., allocating credits to multiple issues) rather than a simple binary choice.¹⁶

7. Conclusion

The analysis of *E-Voting via Upgradable Smart Contracts, Secured Electronic Voting System, and IPFS-based Scalability* highlights a rapid maturation in blockchain voting research. We have moved from simple ledger recording to sophisticated architectures involving proxies, hybrid storage, and biometric fusion.

- **Paper A** teaches us the importance of **upgradability**; without it, systems are brittle.
- **Paper B** reminds us that **frontend performance** drives adoption.
- **Paper C** demonstrates that **hybrid storage (IPFS)** is the only economic path forward.

However, none of these papers are ready for a high-stakes, adversarial environment. They lack the cryptographic armor to protect against coercion and the privacy architecture to prevent state surveillance.

The **Aegis Protocol** proposed in this report fills these gaps. By synthesizing **MACI** for anti-collusion, **Anon Aadhaar** for privacy, **ERC-4337** for usability, and **ZK-Rollups** for scale, we can construct an election infrastructure that is not only immutable and transparent but also fundamentally secure against the nuanced threats of the modern era. This represents the

shift from "digitalizing the ballot box" to "cryptographically guaranteeing democracy."

Works cited

1. E-Voting_via_Upgradable_Smart_Contracts_on_Blockchain.pdf
2. Defending Vote Casting: Using Blockchain-based Mobile Voting Applications in Government Elections | The Belfer Center for Science and International Affairs, accessed on February 6, 2026, <https://www.belfercenter.org/publication/defending-vote-casting-using-blockchain-based-mobile-voting-applications-government>
3. Blockchain and E-Voting Systems: A Review of Progress and Research Opportunities, accessed on February 6, 2026, https://ijsret.com/wp-content/uploads/2025/01/IJSRET_V11_issue1_246.pdf
4. Blockchain-Based E-Voting Systems: A Technology Review - MDPI, accessed on February 6, 2026, <https://www.mdpi.com/2079-9292/13/1/17>
5. Upgradeable Smart Contracts: Proxies, Patterns, Pitfalls and CI/CD Safeguards, accessed on February 6, 2026, <https://www.octane.security/post/upgradeable-smart-contracts-proxies-patterns-pitfalls-cicd-safeguards>
6. (PDF) Siva - The IPFS Search Engine - ResearchGate, accessed on February 6, 2026, https://www.researchgate.net/publication/338649397_Siva_-_The_IPFS_Search_Engine
7. MACI Report - HackMD, accessed on February 6, 2026, <https://hackmd.io/9cVvR0cdRvSEC6CV5zZHqQ>
8. Minimal Anti-Collusion Infrastructure (MACI) | MACI, accessed on February 6, 2026, <https://maci.pse.dev/>
9. Anon Aadhaar | PSE, accessed on February 6, 2026, <https://pse.dev/projects/anon-aadhaar>
10. Anon aadhaar: private identity, accessed on February 6, 2026, <https://anon-aadhaar.pse.dev/learn>
11. How to Develop a Blockchain Application: A Beginner's Guide - Webissoft, accessed on February 6, 2026, <https://webissoft.com/articles/develop-blockchain-application/>
12. Paymasters - ERC-4337 Documentation, accessed on February 6, 2026, <https://docs.erc4337.io/paymasters/index.html>
13. Layer-2 Scaling Solutions for Blockchain: An Expert's Guide - OnGraph, accessed on February 6, 2026, <https://www.ongraph.com/layer-2-scaling-solutions-for-blockchain-an-experts-guide/>
14. Ethereum Layer-2 Solutions Guide 2024: Polygon vs Optimism vs Arbitrum, accessed on February 6, 2026, <https://www.rapidinnovation.io/post/polygon-vs-optimism-vs-arbitrum-comparing-ethereum-layer-2-solutions>
15. What is Soulbound Token (SBT)? Guide to Web3 Identity | Cube Exchange,

accessed on February 6, 2026,

<https://www.cube.exchange/what-is/soulbound-token>

16. Quadratic Voting: A How-To Guide | Gitcoin Blog, accessed on February 6, 2026,

<https://www.gitcoin.co/blog/quadratic-voting-a-how-to-guide>